OOP (JAVA) AND COMPUTER GRAPHICS

Question Bank with Answers

Model answers, Java programs, algorithms with worked examples and comparison tables for exam preparation

Unit	Topic	Questions
I	Introduction to OOP Concepts and Control Structures	13
II	Classes, Objects and Arrays	16
III	Inheritance, Polymorphism, Exception Handling and Multithreading	17
IV	Graphics Primitives, Scan Conversion, Windowing and Clipping	17
V	2D and 3D Transformations and Projections	9
	Total	**72**

Programs are written in Java (version 8 or later). Graphics algorithms are given in C-style pseudocode.

Join Telegram: t.me/SPPU_TE_BE_COMP

PDF BY VL







✨ WHY JOIN ?

🏆 Trusted by 80,000+ Engineering Students
🔔 Stay Updated with latest university news
🚀 Make your engineering journey easy with us

Best Regards, Vishwajeet Londhe

© 2026 Vishwajeet Londhe. All Rights Reserved.

UNIT I - Introduction to OOP Concepts and Control Structures

<table>
<tr><th>S.No</th><th>Question</th><th>Answer</th></tr>
<tr><td>1</td><td>What is a programming paradigm? Explain the different types of programming paradigms.</td><td>A programming paradigm is a style or approach to organising a program, that is, the way a problem is modelled and its solution is expressed in code. A language may support one or several paradigms.
<table>
<tr><th>Paradigm</th><th>Main idea</th><th>Example languages</th></tr>
<tr><td>Procedural (imperative)</td><td>Program is a sequence of statements grouped into procedures/functions that work on shared data</td><td>C, Pascal, FORTRAN</td></tr>
<tr><td>Object-oriented</td><td>Program is a collection of objects that combine data and behaviour and communicate by messages</td><td>Java, C++, C#, Python</td></tr>
<tr><td>Functional</td><td>Computation is the evaluation of mathematical functions; avoids changing state</td><td>Haskell, Lisp, Scala</td></tr>
<tr><td>Logic / rule-based</td><td>Program is a set of facts and rules; the system infers answers</td><td>Prolog, CLIPS</td></tr>
<tr><td>Event-driven</td><td>Flow is decided by events such as clicks and messages</td><td>JavaScript, Visual Basic</td></tr>
<tr><td>Concurrent / parallel</td><td>Several computations run at the same time</td><td>Go, Erlang, Java threads</td></tr>
</table></td></tr>
<tr><td>2</td><td>Explain the difference between procedural, object-oriented, functional and logic/rule-based paradigms.</td><td>
<table>
<tr><th>Aspect</th><th>Procedural</th><th>Object-oriented</th><th>Functional</th><th>Logic / rule</th></tr>
<tr><td>Basic unit</td><td>Procedure</td><td>Class and object</td><td>Function</td><td>Fact and rule</td></tr>
<tr><td>Data and code</td><td>Kept separate</td><td>Bundled together</td><td>Data is immutable</td><td>Stored as facts</td></tr>
<tr><td>State</td><td>Global/shared data</td><td>Hidden inside objects</td><td>Avoided</td><td>Knowledge base</td></tr>
<tr><td>Control</td><td>Top-down steps</td><td>Message passing</td><td>Function composition</td><td>Inference engine</td></tr>
<tr><td>Reuse</td><td>Functions</td><td>Inheritance, composition</td><td>Higher-order functions</td><td>Rules</td></tr>
<tr><td>Example</td><td>C</td><td>Java</td><td>Haskell</td><td>Prolog</td></tr>
</table></td></tr>
<tr><td>3</td><td>Why do we need Object-Oriented Programming (OOP)?</td><td>

In procedural programs data is global and any function can change it, which makes large programs hard to debug and maintain. OOP hides data inside objects and exposes it only through methods.
It models real-world entities (student, account, vehicle) directly, so the design is closer to the problem.
Inheritance and polymorphism allow existing code to be reused and extended instead of rewritten.
Programs are built from independent classes, so teams can develop and test parts separately and changes stay local.
</td></tr>
</table>

S.No	Question	Answer

4	**Explain the main features of OOP: classes and objects, data abstraction and encapsulation, inheritance, polymorphism.**	• **Class and object:** a class is a blueprint that defines data (fields) and behaviour (methods); an object is a real instance of the class with its own state.
		• **Abstraction:** showing only the essential features and hiding the details. A driver uses the steering wheel without knowing the engine.
		• **Encapsulation:** wrapping data and the methods that work on it into one unit (a class) and restricting direct access using `private`.
		• **Inheritance:** a new class (subclass) acquires the fields and methods of an existing class (superclass).
		• **Polymorphism:** one name, many forms: the same method call behaves differently depending on the object or the arguments.
		`class Animal { // class`
		`  private String name; // encapsulated data`
		`  Animal(String n) { name = n; }`
		`  String getName() { return name; }`
		`  void sound() { System.out.println("..."); }`
		`}`
		`class Dog extends Animal { // inheritance`
		`  Dog(String n) { super(n); }`
		`  @Override void sound() { System.out.println("Bark"); } // polymorphism`
		`}`

5	**What are the benefits of OOP?**	• **Reusability** through inheritance and composition. • **Security** because data hiding prevents accidental modification. • **Maintainability:** a change inside a class does not affect users of its public interface. • **Modularity** and easy team development; classes can be tested independently. • **Extensibility:** new object types can be added with little change to existing code. • A natural mapping of real-world problems to software, which makes the design easy to understand.
6	**Explain Java as an object-oriented programming language.**	Java was designed from the start around classes and objects: all code lives inside classes, and the language directly supports encapsulation, inheritance, polymorphism and abstraction (abstract classes and interfaces). Every class implicitly extends `java.lang.Object`. • **Platform independent:** source is compiled to bytecode that runs on any JVM (write once, run anywhere). • **Robust and secure:** strong typing, exception handling, automatic garbage collection, no explicit pointers. • **Multithreaded** with built-in thread support, and **distributed** with a rich networking library. • Java is *not 100 percent pure* object oriented because the eight primitive types (int, char, ...) are not objects, although wrapper classes such as Integer are provided.

<table>
<tr><th>S.No</th><th>Question</th><th>Answer</th></tr>
<tr><td>7</td><td>Write the structure of a simple Java program and explain each section.</td><td>
<pre>/* Documentation section: comments about the program */
package demo;   // package statement (optional)
import java.util.Scanner;   // import statements (optional)

public class Hello {   // class definition
    static int count = 0;   // fields (class-level variables)

    public static void main(String[] args) {   // main method
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter name: ");
        String name = sc.nextLine();
        System.out.println("Hello, " + name);
    }
}</pre>
<table>
<tr><th>Section</th><th>Purpose</th></tr>
<tr><td>Documentation</td><td>Comments (//, /* */, /** */) describing the program</td></tr>
<tr><td>package</td><td>Names the package the class belongs to; must be the first statement</td></tr>
<tr><td>import</td><td>Brings classes from other packages into scope</td></tr>
<tr><td>Class definition</td><td>Every Java program has at least one class; the file name must match the public class</td></tr>
<tr><td>main method</td><td>Entry point: public static void main(String[] args)</td></tr>
<tr><td>Statements</td><td>The instructions executed when the program runs</td></tr>
</table>
</td></tr>
<tr><td>8</td><td>Explain the role of the JVM (Java Virtual Machine) in Java program execution.</td><td>
The Java compiler (<code>javac</code>) converts <code>Hello.java</code> into platform-neutral bytecode (<code>Hello.class</code>). The JVM, which is specific to each operating system, loads and runs this bytecode, so the same class file works everywhere.
<pre>Hello.java --javac--> Hello.class (bytecode) --JVM--> machine code
                                    |
                    Class loader -> Bytecode verifier -> Interpreter / JIT
                                    |
                        Runtime memory: heap, stack, method area</pre>

Class loader loads .class files into memory.
Bytecode verifier checks that the code is safe and does not break access rules.
Interpreter and JIT compiler execute bytecode; the Just-In-Time compiler converts frequently used code to native machine code for speed.
Garbage collector frees memory of objects that are no longer referenced.
JDK = JRE + development tools (javac, debugger); JRE = JVM + class library.

</td></tr>
<tr><td>9</td><td>Differentiate between primitive types and reference types in Java.</td><td>
<table>
<tr><th>Primitive types</th><th>Reference types</th></tr>
<tr><td>Store the actual value</td><td>Store the address (reference) of an object</td></tr>
<tr><td>Fixed size, defined by the language</td><td>Object size depends on the class</td></tr>
<tr><td>8 types: byte, short, int, long, float, double, char, boolean</td><td>Classes, interfaces, arrays, enums, String</td></tr>
<tr><td>Default value 0, 0.0, false</td><td>Default value null</td></tr>
<tr><td>Passed to methods by value (a copy)</td><td>A copy of the reference is passed, so the object can be modified</td></tr>
<tr><td>Kept on the stack (as locals)</td><td>Object lives on the heap</td></tr>
</table>
<table>
<tr><th>Type</th><th>Size</th><th>Type</th><th>Size</th></tr>
<tr><td>byte</td><td>8 bits</td><td>float</td><td>32 bits</td></tr>
<tr><td>short</td><td>16 bits</td><td>double</td><td>64 bits</td></tr>
<tr><td>int</td><td>32 bits</td><td>char</td><td>16 bits (Unicode)</td></tr>
<tr><td>long</td><td>64 bits</td><td>boolean</td><td>true / false</td></tr>
</table>
</td></tr>
</table>

<table>
<tr><th>S.No</th><th>Question</th><th>Answer</th></tr>
<tr><td>10</td><td>Explain operators and expressions in Java with examples.</td><td>An operator performs an operation on operands; an expression is a combination of variables, constants and operators that evaluates to a value.
<table>
<tr><th>Category</th><th>Operators</th><th>Example</th></tr>
<tr><td>Arithmetic</td><td>+ - * / %</td><td>7 / 2 = 3, 7 % 2 = 1</td></tr>
<tr><td>Relational</td><td>== != > < >= <=</td><td>5 > 3 is true</td></tr>
<tr><td>Logical</td><td>&& || !</td><td>(a > 0) && (b > 0)</td></tr>
<tr><td>Assignment</td><td>= += -= *= /= %=</td><td>x += 5 means x = x + 5</td></tr>
<tr><td>Unary</td><td>+ - ++ -- !</td><td>i++ (post), ++i (pre)</td></tr>
<tr><td>Bitwise</td><td>& | ^ ~ << >> >>></td><td>5 & 3 = 1, 5 << 1 = 10</td></tr>
<tr><td>Ternary</td><td>cond ? a : b</td><td>max = (x > y) ? x : y</td></tr>
<tr><td>instanceof</td><td>obj instanceof Class</td><td>s instanceof String</td></tr>
</table>
<pre>
int a = 10, b = 3;
int sum = a + b;   // 13
double avg = (a + b) / 2.0; // 6.5 (2.0 forces floating-point division)
boolean ok = (a > b) && (b != 0);   // true
int big = (a > b) ? a : b; // 10
int i = 5; int j = i++ + ++i;   // 5 + 7 = 12, i becomes 7
</pre>
Precedence (high to low): postfix/unary, * / %, + -, shifts, relational, equality, &, ^, |, &&, ||, ternary, assignment. Use brackets to make the order clear.</td></tr>
<tr><td>11</td><td>What is the Java Class Library?</td><td>The Java Class Library (the Java API) is the large collection of ready-made, tested classes and interfaces that ship with the JDK, organised into packages. It saves the programmer from writing common functionality from scratch.
<table>
<tr><th>Package</th><th>Contents</th></tr>
<tr><td>java.lang</td><td>Core classes (Object, String, Math, wrappers, Thread); imported automatically</td></tr>
<tr><td>java.util</td><td>Collections (ArrayList, HashMap), Scanner, Random, Date</td></tr>
<tr><td>java.io / java.nio</td><td>Input and output streams, files</td></tr>
<tr><td>java.net</td><td>Sockets and URLs</td></tr>
<tr><td>java.sql</td><td>JDBC database access</td></tr>
<tr><td>java.awt / javax.swing</td><td>Graphical user interface</td></tr>
<tr><td>java.time</td><td>Date and time API</td></tr>
</table></td></tr>
</table>

<table>
<tr><th>S.No</th><th>Question</th><th>Answer</th></tr>
<tr><td>12</td><td>Explain control statements in Java with syntax and examples: if, if-else, nested if; for, while, do-while, for-each; break, continue.</td><td>

```
// ---- Decision making ----
if (marks >= 40) { System.out.println("Pass"); }
else             { System.out.println("Fail"); }

if (m >= 75)      System.out.println("Distinction");   // else-if ladder
else if (m >= 60) System.out.println("First class");
else              System.out.println("Pass");

if (a > 0) {                          // nested if
    if (b > 0) System.out.println("Both positive");
}

// ---- Loops ----
for (int i = 1; i <= 5; i++) System.out.print(i + " ");   // 1 2 3 4 5

int n = 5;
while (n > 0) { System.out.print(n + " "); n--; }          // 5 4 3 2 1

int k = 10;
do { System.out.println(k); k++; } while (k < 5);  // runs once: 10

int[] arr = {3, 6, 9};
for (int x : arr) System.out.print(x + " ");      // for-each: 3 6 9

// ---- Jump statements ----
for (int i = 1; i <= 10; i++) {
    if (i == 4) continue;     // skip 4
    if (i == 7) break;        // leave the loop
    System.out.print(i + " ");                    // 1 2 3 5 6
}
```

- **if / if-else:** chooses one path using a boolean condition.
- **while** tests the condition first (entry-controlled); **do-while** tests it after the body, so the body runs *at least once* (exit-controlled).
- **for-each** visits every element of an array or collection without an index.
- **break** exits the nearest loop or switch; **continue** skips to the next iteration.

</td></tr>
<tr><td>13</td><td>Case study: compare Java's OOP features with another OOP language (C++ or Python).</td><td>

Feature	Java	C++	Python
Execution	Compiled to bytecode, run on JVM	Compiled to native code	Interpreted (bytecode on a VM)
Typing	Static, strong	Static	Dynamic, strong
Multiple inheritance	Only through interfaces	Supported for classes	Supported for classes
Pointers	No explicit pointers; references only	Full pointer support	No pointers
Memory management	Automatic garbage collection	Manual (new/delete)	Automatic (reference counting + GC)
Operator overloading	Not supported (except + for String)	Supported	Supported (dunder methods)
Everything is an object?	No; primitives exist	No	Yes
Platform independence	Yes, via JVM	No; recompile per platform	Yes, via interpreter
Access control	private, protected, public, default	private, protected, public	By convention (_name)

Conclusion: Java removes the dangerous parts of C++ (pointers, manual memory, multiple class inheritance) for safety and portability, while Python favours brevity and flexibility at the cost of speed and compile-time checking.

</td></tr>
</table>

UNIT II - Classes, Objects and Arrays

<table>
<tr><th>S.No</th><th>Question</th><th>Answer</th></tr>
<tr><td>1</td><td>What is a class and what is an object?</td><td>
<table>
<tr><th>Class</th><th>Object</th></tr>
<tr><td>A blueprint or template that defines fields and methods</td><td>A real instance created from the class</td></tr>
<tr><td>Logical entity; no memory for field values until an object is made</td><td>Physical entity; occupies memory on the heap</td></tr>
<tr><td>Declared once</td><td>Many objects can be created from one class</td></tr>
<tr><td>Example: Student</td><td>Example: Student s1 = new Student();</td></tr>
</table>
</td></tr>
<tr><td>2</td><td>Explain field declaration and method declaration with examples.</td><td>Fields (instance variables) store the state of an object. Methods define its behaviour.
<pre>class Student {
    // field declarations: [modifier] type name [= value];
    private int rollNo;
    private String name;
    static String college = "ABC College";   // class-level field

    // method declaration:
    // [modifier] returnType name(parameterList) { body }
    public void setData(int r, String n) {
        rollNo = r;
        name = n;
    }
    public void display() {
        System.out.println(rollNo + " " + name + " " + college);
    }
    public int square(int x) { return x * x; }
}</pre>Fields get default values automatically (0, 0.0, false, null); local variables do not and must be initialised before use.</td></tr>
<tr><td>3</td><td>How do you create and access class members?</td><td>Objects are created with new; members are accessed with the dot operator. Static members are accessed through the class name.
<pre>Student s1 = new Student();   // declaration + creation
s1.setData(101, "Amit");   // call an instance method
s1.display();   // 101 Amit ABC College
System.out.println(Student.college);   // static member via class

Student s2 = s1;   // copies the reference, not the object
Student s3 = new Student();   // a different object</pre>new allocates memory on the heap, calls the constructor and returns the reference. Private members are accessible only inside the class, so outside code must use public methods.</td></tr>
<tr><td>4</td><td>Explain how to pass objects as parameters and return objects from methods.</td><td>Java always passes arguments by value. For an object, the value that is copied is the reference, so the method can change the object's contents, but it cannot make the caller's variable point to a different object.
<pre>class Point {
    int x, y;
    Point(int x, int y) { this.x = x; this.y = y; }

    Point add(Point p) {   // object as parameter
        return new Point(x + p.x, y + p.y);   // object returned
    }
    static void move(Point p) { p.x += 10; } // changes caller's object
    static void reset(Point p) { p = new Point(0, 0); } // no effect outside
}
Point a = new Point(1, 2), b = new Point(3, 4);
Point c = a.add(b);   // c = (4, 6)
Point.move(a);   // a.x becomes 11
Point.reset(a);   // a is unchanged: still (11, 2)</pre></td></tr>
</table>

S.No	Question	Answer

5	**What are set and get methods? Give an example.**	Setters and getters (mutator and accessor methods) provide **controlled access** to private fields. The setter can validate the value; the getter hides how the data is stored. This is encapsulation in practice.
		`class Account {`
		`  private double balance;`
		
		`  public double getBalance() { return balance; }`
		
		`  public void setBalance(double b) {`
		`  if (b >= 0) balance = b;`
		`  else System.out.println("Balance cannot be negative");`
		`  }`
		`}`
		`Account a = new Account();`
		`a.setBalance(5000);`
		`System.out.println(a.getBalance());  // 5000.0`

6	**Explain constructors and the use of the this keyword.**	A **constructor** is a special method with the **same name as the class and no return type**. It runs automatically when an object is created and is used to initialise fields. If none is written, the compiler supplies a default no-argument constructor.
		`class Box {`
		`  int l, w, h;`
		`  Box() { this(1, 1, 1); }  // calls the next constructor`
		`  Box(int l, int w, int h) {  // parameterised`
		`  this.l = l;  this.w = w;  this.h = h;  // this: field vs parameter`
		`  }`
		`  Box(Box other) { this(other.l, other.w, other.h); }  // copy constructor`
		`  Box self() { return this; }  // returns the current object`
		`}`
		Uses of this: (1) refer to the current object's field when a parameter has the same name; (2) call another constructor of the same class (must be the first statement); (3) pass or return the current object; (4) call another method of the same object.

7	**What are static methods and variables?**	A **static** member belongs to the **class**, not to any one object. There is only one copy, shared by all objects, and it exists even before any object is created.
		`class Counter {`
		`  static int count = 0;  // shared by all objects`
		`  Counter() { count++; }`
		`  static int getCount() { return count; }  // static method`
		`}`
		`new Counter(); new Counter(); new Counter();`
		`System.out.println(Counter.getCount());  // 3`
		• A static method can use only static members directly; it has no `this` and cannot call instance members without an object.
		• `main` is static so that the JVM can call it without creating an object.
		• A `static { }` block runs once when the class is loaded.

8	**Explain method overloading with an example.**	Method overloading means having **several methods with the same name in one class but different parameter lists** (different number, type or order of parameters). The return type alone cannot distinguish overloaded methods. The compiler picks the version at compile time, so it is *compile-time polymorphism*.
		`class Calc {`
		`  int add(int a, int b)  { return a + b; }`
		`  int add(int a, int b, int c)  { return a + b + c; }`
		`  double add(double a, double b)  { return a + b; }`
		`}`
		`Calc c = new Calc();`
		`c.add(2, 3);  // 5  (int, int)`
		`c.add(1, 2, 3);  // 6  (three ints)`
		`c.add(2.5, 1.5);  // 4.0  (double, double)`

<table>
<tr><th>S.No</th><th>Question</th><th>Answer</th></tr>
<tr><td>9</td><td>What is the scope of a declaration in Java?</td><td>Scope is the region of the program in which a name (variable, method) is visible.
<table>
<tr><th>Kind</th><th>Scope</th><th>Lifetime</th></tr>
<tr><td>Instance field</td><td>Whole class, through the object</td><td>As long as the object exists</td></tr>
<tr><td>Static field</td><td>Whole class</td><td>Until the class is unloaded</td></tr>
<tr><td>Parameter</td><td>The method body</td><td>During the method call</td></tr>
<tr><td>Local variable</td><td>From its declaration to the end of its block { }</td><td>Until the block ends</td></tr>
<tr><td>Loop variable</td><td>Only inside the for statement and body</td><td>Until the loop ends</td></tr>
</table>
<pre>class Demo {
    int x = 5;   // class scope
    void f(int p) {   // p: method scope
        int y = 1;   // local
        { int z = 2; }   // z is gone after this block
        // System.out.println(z);   // error: z not visible
        int x = 9;   // local x hides the field
        System.out.println(this.x); // 5, field reached via this
    }
}</pre>
Java does not allow a local variable to be redeclared inside a nested block of the same method, but a local variable may shadow a field.</td></tr>
<tr><td>10</td><td>What is a Java API package? Give examples.</td><td>A package is a named group of related classes and interfaces. It avoids name clashes and controls access. The standard library is itself a set of packages, called the Java API. Use <code>import</code> to bring a class into scope; <code>java.lang</code> is imported automatically.

java.lang - String, Math, Integer, Thread, System
java.util - ArrayList, HashMap, Scanner, Arrays, Random
java.io - File, FileReader, PrintWriter, BufferedReader
java.sql - Connection, Statement, ResultSet
java.awt / javax.swing - GUI components

<pre>import java.util.ArrayList;   // one class
import java.io.*;   // all classes of the package</pre>
A user-defined package is created with <code>package mypack;</code> at the top of the file.</td></tr>
<tr><td>11</td><td>Explain arrays in Java: declaration, creation and initialization.</td><td>An array is a fixed-size, indexed collection of elements of the same type. Arrays are objects, indexed from 0, and their length is available as <code>arr.length</code>.
<pre>int[] a;   // declaration (also: int a[];)
a = new int[5];   // creation: 5 elements, all 0
int[] b = new int[3];   // declaration + creation
int[] c = {10, 20, 30};   // initialisation with values
b[0] = 7;   // assign using index

for (int i = 0; i < c.length; i++)
    System.out.print(c[i] + " ");   // 10 20 30
for (int v : c) System.out.print(v + " "); // for-each</pre>
Accessing an index outside 0 to length-1 throws <code>ArrayIndexOutOfBoundsException</code>.</td></tr>
</table>

S.No	Question	Answer

12	**Explain multidimensional arrays and variable-length argument lists.**	A multidimensional array is an **array of arrays**. Rows may have different lengths (a jagged array).
		`int[][] m = new int[2][3];  // 2 rows, 3 columns`
		`int[][] t = { {1, 2, 3}, {4, 5, 6} };  // initialised`
		`System.out.println(t[1][2]);  // 6`
		`int[][] jag = new int[3][];  // jagged array`
		`jag[0] = new int[1];  jag[1] = new int[2];  jag[2] = new int[3];`
		`for (int i = 0; i < t.length; i++) {`
		`  for (int j = 0; j < t[i].length; j++)`
		`  System.out.print(t[i][j] + " ");`
		`  System.out.println();`
		`}`
		Variable-length arguments (varargs) let a method accept any number of arguments of one type. Inside the method the parameter is an array. It must be the last parameter, and a method can have only one.
		`static int sum(int... nums) {`
		`  int total = 0;`
		`  for (int n : nums) total += n;`
		`  return total;`
		`}`
		`sum();  // 0`
		`sum(4, 5);  // 9`
		`sum(1, 2, 3, 4);  // 10`

13	**How are command-line arguments used in Java?**	Values typed after the class name when running the program are passed to `main` as the `String[] args` array. They are always strings, so numbers must be converted.
		`public class Add {`
		`  public static void main(String[] args) {`
		`  if (args.length < 2) {`
		`  System.out.println("Usage: java Add num1 num2");`
		`  return;`
		`  }`
		`  int a = Integer.parseInt(args[0]);`
		`  int b = Integer.parseInt(args[1]);`
		`  System.out.println("Sum = " + (a + b));`
		`  }`
		`}`
		`// javac Add.java`
		`// java Add 12 30  -> Sum = 42`

<table>
<tr><th>S.No</th><th>Question</th><th>Answer</th></tr>
<tr><td>14</td><td>Explain input/output streams in Java: byte streams and character streams.</td><td>A stream is a sequence of data flowing from a source (keyboard, file, network) to a program (input) or from a program to a destination (output).<table><tr><th>Byte streams</th><th>Character streams</th></tr><tr><td>Handle raw 8-bit data (images, audio, any file)</td><td>Handle 16-bit Unicode text</td></tr><tr><td>Base classes: InputStream, OutputStream</td><td>Base classes: Reader, Writer</td></tr><tr><td>FileInputStream, FileOutputStream, BufferedInputStream</td><td>FileReader, FileWriter, BufferedReader, PrintWriter</td></tr><tr><td>Read and write with bytes (int 0-255)</td><td>Read and write with chars and lines</td></tr></table><pre>// copy any file with byte streams
try (FileInputStream in = new FileInputStream("a.dat");
     FileOutputStream out = new FileOutputStream("b.dat")) {
    int b;
    while ((b = in.read()) != -1) out.write(b);
} catch (IOException e) { System.out.println(e); }

// read a text file line by line with character streams
try (BufferedReader br = new BufferedReader(new FileReader("t.txt"))) {
    String line;
    while ((line = br.readLine()) != null) System.out.println(line);
} catch (IOException e) { System.out.println(e); }</pre>The three standard streams are <code>System.in</code> (input), <code>System.out</code> and <code>System.err</code> (output).</td></tr>
<tr><td>15</td><td>What is the PrintWriter class used for?</td><td><code>PrintWriter</code> (in <code>java.io</code>) writes formatted text to a character output stream, a file or a byte stream. It offers <code>print()</code>, <code>println()</code> and <code>printf()</code> like <code>System.out</code>, so any destination can be written to in the same convenient way. Its methods do not throw <code>IOException</code>; use <code>checkError()</code> to test for problems.<pre>try (PrintWriter pw = new PrintWriter("report.txt")) {
    pw.println("Student Report");
    pw.printf("%-10s %5d%n", "Amit", 87);
    pw.print("Done");
} catch (FileNotFoundException e) {
    System.out.println("Cannot open file");
}

PrintWriter console = new PrintWriter(System.out, true); // auto flush
console.println("Hello");</pre>Remember to close or flush the writer; otherwise buffered data may not reach the file.</td></tr>
</table>

S.No	Question	Answer
16	**Case study: explain how a Banking System or Library Management System can be implemented using classes, objects and arrays.**	**Design:** identify the entities (Account, Customer / Book, Member), turn each into a class with private fields and public methods, and store many objects in an array that a manager class searches and updates. `class Account { private int accNo; private String holder; private double balance; Account(int no, String h, double b) { accNo = no; holder = h; balance = b; } int getAccNo() { return accNo; } void deposit(double amt) { if (amt > 0) balance += amt; } boolean withdraw(double amt) { if (amt > 0 && amt <= balance) { balance -= amt; return true; } return false; } void show() { System.out.println(accNo + " " + holder + " " + balance); } } class Bank { private Account[] accounts = new Account[100]; // array of objects private int count = 0; void open(Account a) { if (count < accounts.length) accounts[count++] = a; } Account find(int no) { for (int i = 0; i < count; i++) if (accounts[i].getAccNo() == no) return accounts[i]; return null; } void transfer(int from, int to, double amt) { Account f = find(from), t = find(to); if (f != null && t != null && f.withdraw(amt)) t.deposit(amt); } }` **Library system** works the same way: a `Book` class (id, title, isIssued), a `Member` class, and a `Library` class holding `Book[]` with methods `issue()`, `returnBook()` and `search()`. Encapsulation protects the data, the array stores many objects, and a fixed-size array can later be replaced by `ArrayList` for growth.

UNIT III - Inheritance, Polymorphism, Exception Handling and Multithreading

<table>
<tr><th>S.No</th><th>Question</th><th>Answer</th></tr>
<tr><td>1</td><td>What is inheritance? Explain its types with examples.</td><td>Inheritance is the mechanism by which one class (the subclass or child) acquires the fields and methods of another class (the superclass or parent) using the keyword <code>extends</code>. It models an is-a relationship and promotes code reuse.
<pre>class A { void a() { } }
class B extends A { void b() { } }   // single: A -> B
class C extends B { void c() { } }   // multilevel: A -> B -> C
class D extends A { void d() { } }   // hierarchical: A -> B, A -> D

interface P { void p(); }
interface Q { void q(); }
class R implements P, Q {   // multiple (via interfaces)
    public void p() { }
    public void q() { }
}</pre>
<table>
<tr><th>Type</th><th>Meaning</th><th>Java support</th></tr>
<tr><td>Single</td><td>One subclass, one superclass</td><td>Yes</td></tr>
<tr><td>Multilevel</td><td>A chain: grandparent, parent, child</td><td>Yes</td></tr>
<tr><td>Hierarchical</td><td>Many subclasses of one superclass</td><td>Yes</td></tr>
<tr><td>Multiple</td><td>One class with several superclasses</td><td>Only through interfaces; not with classes, to avoid the diamond ambiguity</td></tr>
<tr><td>Hybrid</td><td>Combination of the above</td><td>Through interfaces</td></tr>
</table></td></tr>
<tr><td>2</td><td>What are superclasses and subclasses?</td><td>The superclass is the class being inherited from; the subclass is the class that inherits and may add new members or override inherited ones. The keyword <code>super</code> refers to the superclass: <code>super.method()</code> calls the parent's version and <code>super(args)</code> calls the parent's constructor. A class declared <code>final</code> cannot be a superclass, and every class has <code>Object</code> at the top of its chain.
<pre>class Vehicle {
    int speed = 60;
    void show() { System.out.println("Vehicle"); }
}
class Car extends Vehicle {
    int speed = 120;
    void show() {
        super.show();   // Vehicle
        System.out.println(super.speed + " " + speed); // 60 120
    }
}</pre></td></tr>
<tr><td>3</td><td>What is the use of the protected keyword?</td><td><code>protected</code> gives access to the members of a class from the same package and from subclasses, even when the subclass is in a different package. It lets a parent expose internals to its children without making them public.
<table>
<tr><th>Modifier</th><th>Same class</th><th>Same package</th><th>Subclass (other package)</th><th>Anywhere</th></tr>
<tr><td>private</td><td>Yes</td><td>No</td><td>No</td><td>No</td></tr>
<tr><td>default (none)</td><td>Yes</td><td>Yes</td><td>No</td><td>No</td></tr>
<tr><td>protected</td><td>Yes</td><td>Yes</td><td>Yes</td><td>No</td></tr>
<tr><td>public</td><td>Yes</td><td>Yes</td><td>Yes</td><td>Yes</td></tr>
</table></td></tr>
</table>

<table>
<tr><th>S.No</th><th>Question</th><th>Answer</th></tr>
<tr><td>4</td><td>How are constructors used in subclasses?</td><td>Constructors are not inherited. When a subclass object is created, the superclass constructor runs first, so the parent part of the object is built before the child part. The first line of a subclass constructor is an implicit <code>super()</code> unless the programmer writes <code>super(args)</code> (or <code>this(...)</code>) explicitly. If the parent has no no-argument constructor, the child must call a parent constructor explicitly.

<pre>class Person {
    Person(String n) { System.out.println("Person " + n); }
}
class Student extends Person {
    Student(String n, int r) {
        super(n);   // must be first statement
        System.out.println("Student " + r);
    }
}
new Student("Asha", 7);
// Output: Person Asha
//   Student 7</pre></td></tr>
<tr><td>5</td><td>What is polymorphism? Explain compile-time and run-time polymorphism with examples.</td><td>Polymorphism means 'many forms': the same name or call produces different behaviour in different situations.

<table>
<tr><th>Compile-time (static)</th><th>Run-time (dynamic)</th></tr>
<tr><td>Achieved by method overloading</td><td>Achieved by method overriding</td></tr>
<tr><td>Method chosen by the compiler from the argument types</td><td>Method chosen by the JVM from the actual object type</td></tr>
<tr><td>Same class, different parameters</td><td>Subclass redefines a superclass method with the same signature</td></tr>
<tr><td>Also called early binding</td><td>Also called late binding</td></tr>
</table>
<pre>class Shape {
    void draw() { System.out.println("Shape"); }
}
class Circle extends Shape {
    @Override void draw() { System.out.println("Circle"); }
}
class Square extends Shape {
    @Override void draw() { System.out.println("Square"); }
}
Shape s;
s = new Circle(); s.draw();   // Circle   (run time decision)
s = new Square(); s.draw();   // Square

class Print {   // compile-time: overloading
    void show(int i)   { System.out.println("int " + i); }
    void show(String s) { System.out.println("String " + s); }
}</pre></td></tr>
<tr><td>6</td><td>What are abstract classes and abstract methods?</td><td>An abstract method has a declaration but no body; it must be implemented by a subclass. A class containing one or more abstract methods must be declared <code>abstract</code>. An abstract class cannot be instantiated, but it can have fields, constructors and ordinary methods, and it acts as a common base that defines a contract.

<pre>abstract class Shape {
    abstract double area();   // no body
    void info() { System.out.println("Area = " + area()); } // concrete
}
class Rect extends Shape {
    double l, b;
    Rect(double l, double b) { this.l = l; this.b = b; }
    double area() { return l * b; }   // must implement
}
// Shape s = new Shape();   // error: cannot instantiate
Shape s = new Rect(4, 5);
s.info();   // Area = 20.0</pre>
A subclass that does not implement all abstract methods must itself be declared abstract.</td></tr>
</table>

<table>
<tr><th>S.No</th><th>Question</th><th>Answer</th></tr>
<tr><td>7</td><td>What are final methods and final classes?</td><td>

final variable: value cannot change once assigned (a constant).
final method: cannot be overridden by a subclass. Use it to protect critical behaviour.
final class: cannot be extended. Example: <code>String</code>, <code>Integer</code> and other wrapper classes.

<pre>final class Constants { }   // class Sub extends Constants -> error

class Bank {
    final double RATE = 0.04;   // constant
    final void audit() { }   // cannot be overridden
}
class Sbi extends Bank {
    // void audit() { }   // compile error
}</pre>
Final methods can also be inlined by the compiler, and final classes help make a class immutable and secure.
</td></tr>
<tr><td>8</td><td>What is dynamic binding? Explain with an example.</td><td>
Dynamic binding (late binding, dynamic method dispatch) means that the method to be executed is decided at run time from the actual object that a superclass reference points to, not from the reference type. It works for overridden instance methods. Static, private and final methods and fields are bound at compile time.
<pre>class Animal {
    String name = "Animal";
    void sound() { System.out.println("Some sound"); }
}
class Dog extends Animal {
    String name = "Dog";
    @Override void sound() { System.out.println("Bark"); }
}
class Cat extends Animal {
    @Override void sound() { System.out.println("Meow"); }
}
Animal a = new Dog();
a.sound();   // Bark   (decided at run time)
System.out.println(a.name);   // Animal (fields are not polymorphic)
a = new Cat();
a.sound();   // Meow</pre>
Benefit: one piece of code (<code>a.sound()</code>) works for every present and future subclass, which is the basis of extensible design.
</td></tr>
<tr><td>9</td><td>What is an interface? How is it different from a class?</td><td>
An interface is a fully abstract contract: it declares methods that implementing classes must provide. Its fields are implicitly <code>public static final</code> and its methods implicitly <code>public abstract</code> (since Java 8 it may also have <code>default</code> and <code>static</code> methods). A class uses <code>implements</code> and may implement many interfaces.
<pre>interface Drawable {
    double PI = 3.14159;   // constant
    void draw();   // abstract
    default void hello() { System.out.println("Drawable"); }
}
class Circle implements Drawable {
    public void draw() { System.out.println("Drawing circle"); }
}</pre>
<table>
<tr><th>Class</th><th>Interface</th></tr>
<tr><td>Can be instantiated (unless abstract)</td><td>Cannot be instantiated</td></tr>
<tr><td>Has fields, constructors, concrete methods</td><td>Constants and method declarations; no constructors</td></tr>
<tr><td>A class extends only one class</td><td>A class can implement many interfaces</td></tr>
<tr><td>Uses extends</td><td>Uses implements (an interface extends other interfaces)</td></tr>
<tr><td>Describes what an object is</td><td>Describes what an object can do</td></tr>
</table>
</td></tr>
</table>

<table>
<tr><th>S.No</th><th>Question</th><th>Answer</th></tr>
<tr><td>10</td><td>What is exception handling? Explain the need for it.</td><td>An exception is an event that occurs at run time and disrupts the normal flow of the program (divide by zero, file not found, invalid index). Exception handling is the mechanism to detect and respond to such errors so that the program does not crash.
Separates error-handling code from normal logic, which keeps the program readable.
Allows the program to recover or terminate gracefully instead of abruptly.
Errors can be propagated up the call stack to a method that knows how to handle them.
Resources (files, connections) can be released reliably with <code>finally</code>.
<pre>Throwable
 |-- Error   (serious JVM problems: OutOfMemoryError)
 |-- Exception
      |-- IOException, SQLException   (checked)
      |-- RuntimeException   (unchecked)
            |-- ArithmeticException, NullPointerException,
                ArrayIndexOutOfBoundsException, NumberFormatException</pre></td></tr>
<tr><td>11</td><td>Explain the keywords try, catch, throw, throws and finally.</td><td><table><tr><th>Keyword</th><th>Purpose</th></tr><tr><td>try</td><td>Block containing code that may cause an exception</td></tr><tr><td>catch</td><td>Block that handles a specific exception type thrown in the try block</td></tr><tr><td>throw</td><td>Explicitly throws one exception object: throw new Exception("msg");</td></tr><tr><td>throws</td><td>In a method header; declares the checked exceptions the method may pass to its caller</td></tr><tr><td>finally</td><td>Always executes (whether or not an exception occurred); used for clean-up</td></tr></table><pre>static void check(int age) throws Exception {   // declares
    if (age < 18) throw new Exception("Not eligible"); // throws
    System.out.println("Eligible");
}
public static void main(String[] args) {
    try {
        check(15);
    } catch (Exception e) {
        System.out.println("Caught: " + e.getMessage()); // Not eligible
    } finally {
        System.out.println("Done");   // always printed
    }
}</pre></td></tr>
</table>

<table>
<tr><th>S.No</th><th>Question</th><th>Answer</th></tr>
<tr><td>12</td><td>Write a program to demonstrate multiple catch and nested try blocks.</td><td>

```
public class ExceptionDemo {
    public static void main(String[] args) {
        int[] a = {10, 20, 30};
        try {                                   // outer try
            try {                               // nested (inner) try
                int r = 10 / 0;                 // ArithmeticException
            } catch (ArithmeticException e) {
                System.out.println("Inner: " + e.getMessage());
            }
            System.out.println(a[5]);           // ArrayIndexOutOfBounds
            System.out.println("not reached");
        } catch (ArrayIndexOutOfBoundsException e) {   // multiple catch
            System.out.println("Outer: " + e.getMessage());
        } catch (NullPointerException e) {
            System.out.println("Null reference");
        } catch (Exception e) {                 // general handler last
            System.out.println("Other: " + e);
        } finally {
            System.out.println("finally always runs");
        }
    }
}
/* Output:
   Inner: / by zero
   Outer: Index 5 out of bounds for length 3
   finally always runs                                  */
```

- The inner catch handles the ArithmeticException, so the outer code continues.
- The exception in the outer part is matched against the catch blocks from top to bottom; the first match runs.
- Catch blocks must go from **specific to general**; a superclass catch placed first makes the later ones unreachable and gives a compile error.

</td></tr>
<tr><td>13</td><td>What are built-in exceptions in Java?</td><td>These are exception classes provided by the Java library for common error conditions.
<table>
<tr><th>Type</th><th>Meaning</th><th>Examples</th></tr>
<tr><td>Checked</td><td>Checked by the compiler; must be caught or declared with throws</td><td>IOException, FileNotFoundException, SQLException, ClassNotFoundException, InterruptedException</td></tr>
<tr><td>Unchecked (runtime)</td><td>Subclasses of RuntimeException; not forced by the compiler</td><td>ArithmeticException, NullPointerException, ArrayIndexOutOfBoundsException, NumberFormatException, ClassCastException, IllegalArgumentException</td></tr>
<tr><td>Errors</td><td>Serious problems the program normally cannot recover from</td><td>OutOfMemoryError, StackOverflowError</td></tr>
</table>
Programmers can also create user-defined exceptions by extending <code>Exception</code> or <code>RuntimeException</code>.</td></tr>
</table>

<table>
<tr><th>S.No</th><th>Question</th><th>Answer</th></tr>
<tr><td>14</td><td>Explain the concept of multithreading.</td><td>A thread is the smallest unit of execution within a program. Multithreading means running two or more threads concurrently inside one process, each following its own path of execution while sharing the same memory.<table><tr><th>Process</th><th>Thread</th></tr><tr><td>Independent program with its own memory space</td><td>Lightweight unit inside a process, shares its memory</td></tr><tr><td>Context switching is costly</td><td>Context switching is cheap</td></tr><tr><td>Communication needs inter-process mechanisms</td><td>Communicates through shared objects</td></tr></table>
Advantages: better CPU utilisation, responsive user interfaces, faster handling of many tasks (servers, downloads), economical resource sharing.
Problems: shared data can become inconsistent (race conditions), so access must be synchronised; deadlock is possible.</td></tr>
<tr><td>15</td><td>How do you create a thread in Java (by extending Thread or implementing Runnable)?</td><td><pre>
// Way 1: extend the Thread class
class MyThread extends Thread {
    public void run() {
        for (int i = 1; i <= 3; i++)
            System.out.println(getName() + " : " + i);
    }
}
// Way 2: implement the Runnable interface
class MyTask implements Runnable {
    public void run() {
        for (int i = 1; i <= 3; i++)
            System.out.println(Thread.currentThread().getName() + ":" + i);
    }
}
public class Test {
    public static void main(String[] args) {
        MyThread t1 = new MyThread();
        Thread t2 = new Thread(new MyTask(), "Worker");
        t1.start();   // start(), not run()
        t2.start();
        new Thread(() -> System.out.println("Lambda")).start();
    }
}
</pre>
<code>start()</code> creates a new thread and calls <code>run()</code> on it; calling <code>run()</code> directly just executes it in the current thread.
Prefer Runnable: Java has single inheritance, so a class that extends Thread cannot extend anything else, while a Runnable class can.
<code>join()</code> waits for a thread to finish; <code>sleep(ms)</code> pauses the current thread.</td></tr>
</table>

<table>
<tr><th>S.No</th><th>Question</th><th>Answer</th></tr>
<tr><td>16</td><td>Explain the Java Thread Model and the Main Thread.</td><td>Java has built-in multithreading support. The JVM's thread scheduler decides which runnable thread gets the CPU, based mainly on priority (1 to 10, default 5) and the operating system's policy.
<pre>
           start()   scheduler picks
   NEW ------------> RUNNABLE <--------------> RUNNING
                       ^ | |
        notify / unlock | | wait(), sleep(), | run() ends
                        | v I/O, lock   v
             WAITING / TIMED_WAITING / BLOCKED   TERMINATED
</pre>
<table>
<tr><th>State</th><th>Meaning</th></tr>
<tr><td>NEW</td><td>Thread object created but start() not yet called</td></tr>
<tr><td>RUNNABLE</td><td>Ready to run or running</td></tr>
<tr><td>BLOCKED</td><td>Waiting to acquire a monitor lock</td></tr>
<tr><td>WAITING / TIMED_WAITING</td><td>Waiting for another thread, or sleeping for a given time</td></tr>
<tr><td>TERMINATED</td><td>run() has completed</td></tr>
</table>
Main thread: when a program starts, the JVM automatically creates one thread, called <code>main</code>, which executes the <code>main()</code> method. All other threads are created from it, and it should normally be the last to finish.
<pre>
public class MainThreadDemo {
    public static void main(String[] args) throws InterruptedException {
        Thread t = Thread.currentThread();
        System.out.println(t);   // name, priority, group
        t.setName("MyMain");
        System.out.println(t.getName()); // MyMain
        for (int i = 3; i > 0; i--) {
            System.out.println(i);
            Thread.sleep(500);
        }
    }
}
</pre></td></tr>
</table>

Join Telegram: t.me/SPPU_TE_BE_COMP

S.No	Question	Answer

17	**Case study: explain how an Online Banking or Library System can be developed using inheritance, exception handling and multithreading.**	**Inheritance:** a general abstract `Account` class holds common data and methods; `SavingsAccount` and `CurrentAccount` extend it and override rules such as interest. **Exception handling:** a custom `InsufficientFundsException` signals business errors. **Multithreading:** each customer session runs in its own thread, and the shared account methods are `synchronized` so simultaneous transactions do not corrupt the balance.
		`class InsufficientFundsException extends Exception {`
		`  InsufficientFundsException(String msg) { super(msg); }`
		`}`
		`abstract class Account {`
		`  protected double balance;`
		`  Account(double b) { balance = b; }`
		`  synchronized void deposit(double amt) { balance += amt; }`
		`  synchronized void withdraw(double amt)`
		`  throws InsufficientFundsException {`
		`  if (amt > balance)`
		`  throw new InsufficientFundsException("Low balance");`
		`  balance -= amt;`
		`  }`
		`  abstract double interestRate();  // overridden below`
		`}`
		`class Savings extends Account {`
		`  Savings(double b) { super(b); }`
		`  double interestRate() { return 0.04; }`
		`}`
		`class Customer extends Thread {  // one thread per user`
		`  private Account acc;  private double amt;`
		`  Customer(String n, Account a, double amt) {`
		`  super(n);  acc = a;  this.amt = amt;`
		`  }`
		`  public void run() {`
		`  try {`
		`  acc.withdraw(amt);`
		`  System.out.println(getName() + " withdrew " + amt);`
		`  } catch (InsufficientFundsException e) {`
		`  System.out.println(getName() + ": " + e.getMessage());`
		`  }`
		`  }`
		`}`
		`// Account a = new Savings(1000);`
		`// new Customer("Ravi", a, 700).start();`
		`// new Customer("Sita", a, 700).start();  // one of them gets 'Low balance'`
		Library system: an abstract `Item` class with subclasses `Book` and `Journal`; a `BookNotAvailableException`; and threads for simultaneous issue/return requests, with synchronised access to the stock count.

UNIT IV - Graphics Primitives, Scan Conversion, Windowing and Clipping

<table>
<tr><th>S.No</th><th>Question</th><th>Answer</th></tr>
<tr><td>1</td><td>What are graphics primitives?</td><td>Graphics primitives are the basic building blocks from which every picture is made. Output primitives describe what is drawn; attributes describe how it looks.
Geometric primitives: point, line, circle, ellipse, polygon, curve.
Attributes: colour, line style and width, fill pattern, text font.
Other primitives: text (character strings) and pixel arrays/images.
Complex objects such as a house or a 3D model are built by combining these primitives.</td></tr>
<tr><td>2</td><td>Define pixel, resolution, aspect ratio and frame buffer.</td><td><table><tr><th>Term</th><th>Definition</th></tr><tr><td>Pixel</td><td>Picture element; the smallest addressable point on the screen, with a position and a colour</td></tr><tr><td>Resolution</td><td>Number of pixels on the screen, written as columns x rows (for example 1920 x 1080); more pixels give a sharper image</td></tr><tr><td>Aspect ratio</td><td>Ratio of the width to the height of the display or image (for example 16:9). Also the ratio of vertical to horizontal points needed for equal-length lines</td></tr><tr><td>Frame buffer</td><td>A region of memory that stores the colour value of every pixel of the image to be displayed; the display controller reads it repeatedly to refresh the screen</td></tr></table>Memory needed = columns x rows x bits per pixel. For example, 1024 x 768 at 24 bits per pixel needs 1024 x 768 x 3 bytes = 2.25 MB.</td></tr>
<tr><td>3</td><td>Explain the different display devices used in computer graphics.</td><td><table><tr><th>Device</th><th>Working principle</th></tr><tr><td>CRT (Cathode Ray Tube)</td><td>An electron gun fires a beam that is deflected by electric or magnetic fields and strikes a phosphor-coated screen, which glows. Needs refreshing</td></tr><tr><td>Raster-scan display</td><td>The beam sweeps the screen line by line, row after row, drawing the picture stored in the frame buffer; typical refresh 60 Hz or more. Used in TVs and monitors</td></tr><tr><td>Random-scan (vector) display</td><td>The beam is steered directly along the lines of the picture, in any order, using a display list. Gives smooth lines but cannot show filled areas well</td></tr><tr><td>LCD</td><td>Liquid crystals between polarisers control how much of a backlight passes through each pixel</td></tr><tr><td>LED / OLED</td><td>LED uses LED backlight or LEDs as pixels; OLED pixels emit their own light, giving deeper blacks and thin screens</td></tr><tr><td>Plasma</td><td>Each cell contains gas that glows when excited by voltage; large wide-angle screens</td></tr></table><table><tr><th>Raster scan</th><th>Random scan</th></tr><tr><td>Picture drawn as pixels in rows</td><td>Picture drawn as lines in any order</td></tr><tr><td>Stored in the frame buffer</td><td>Stored as a display list (commands)</td></tr><tr><td>Good for shaded, filled and realistic images</td><td>Good for line drawings, CAD</td></tr><tr><td>Jagged (aliased) lines</td><td>Smooth lines</td></tr></table></td></tr>
</table>

<table>
<tr><th>S.No</th><th>Question</th><th>Answer</th></tr>
<tr><td>4</td><td>Explain the applications of computer graphics.</td><td>

Computer-aided design (CAD): architecture, mechanical and circuit design.
Entertainment: video games, animation, special effects in films.
Education and training: flight and driving simulators.
Visualisation: scientific, engineering and business data (charts, medical imaging such as CT/MRI).
User interfaces: windows, icons, menus on every modern operating system.
Virtual and augmented reality, GIS and cartography, image processing, e-commerce product views.

</td></tr>
<tr><td>5</td><td>Explain the DDA line drawing algorithm with steps and an example.</td><td>
The Digital Differential Analyzer algorithm draws a line by moving a fixed increment along the major axis and computing the matching increment on the other axis using floating-point arithmetic.

Compute dx = x2 - x1 and dy = y2 - y1.
steps = the larger of |dx| and |dy|.
xinc = dx / steps and yinc = dy / steps.
Plot the start point (x1, y1).
Repeat steps times: x = x + xinc, y = y + yinc, plot (round(x), round(y)).

<pre><code>void ddaLine(int x1, int y1, int x2, int y2) {
    int dx = x2 - x1, dy = y2 - y1;
    int steps = abs(dx) > abs(dy) ? abs(dx) : abs(dy);
    float xinc = (float)dx / steps, yinc = (float)dy / steps;
    float x = x1, y = y1;
    for (int k = 0; k <= steps; k++) {
        putpixel(round(x), round(y), WHITE);
        x += xinc; y += yinc;
    }
}</code></pre>
Example: line from (1, 1) to (6, 4). dx = 5, dy = 3, steps = 5, xinc = 1, yinc = 0.6.
<table>
<tr><th>k</th><th>x</th><th>y (exact)</th><th>Plotted pixel</th></tr>
<tr><td>0</td><td>1</td><td>1.0</td><td>(1, 1)</td></tr>
<tr><td>1</td><td>2</td><td>1.6</td><td>(2, 2)</td></tr>
<tr><td>2</td><td>3</td><td>2.2</td><td>(3, 2)</td></tr>
<tr><td>3</td><td>4</td><td>2.8</td><td>(4, 3)</td></tr>
<tr><td>4</td><td>5</td><td>3.4</td><td>(5, 3)</td></tr>
<tr><td>5</td><td>6</td><td>4.0</td><td>(6, 4)</td></tr>
</table>
Drawbacks: floating-point arithmetic and rounding in every step are slow, and rounding errors can accumulate on long lines.
</td></tr>
</table>

<table>
<tr><th>S.No</th><th>Question</th><th>Answer</th></tr>
<tr><td>6</td><td>Explain Bresenham's line drawing algorithm with an example.</td><td>Bresenham's algorithm uses only integer addition and subtraction. At each step it decides, with a decision parameter p, which of the two candidate pixels is closer to the true line. For a line with slope between 0 and 1 and x1 < x2:
dx = x2 - x1, dy = y2 - y1. Plot (x1, y1).
Initial decision parameter p0 = 2dy - dx.
At each x step: if p < 0, the next pixel is (x+1, y) and p = p + 2dy.
Otherwise the next pixel is (x+1, y+1) and p = p + 2dy - 2dx.
Repeat until x reaches x2.
<pre>void bresenham(int x1, int y1, int x2, int y2) {   // 0 <= slope <= 1
    int dx = x2 - x1, dy = y2 - y1;
    int p = 2 * dy - dx, x = x1, y = y1;
    putpixel(x, y, WHITE);
    while (x < x2) {
        x++;
        if (p < 0) p += 2 * dy;
        else   { y++; p += 2 * dy - 2 * dx; }
        putpixel(x, y, WHITE);
    }
}</pre>Example: (1, 1) to (6, 4): dx = 5, dy = 3, 2dy = 6, 2dy - 2dx = -4, p0 = 6 - 5 = 1.<table><tr><th>Step</th><th>p before</th><th>Decision</th><th>Next pixel</th><th>p after</th></tr><tr><td>0</td><td>1</td><td>p >= 0: x+1, y+1</td><td>(2, 2)</td><td>1 - 4 = -3</td></tr><tr><td>1</td><td>-3</td><td>p < 0: x+1</td><td>(3, 2)</td><td>-3 + 6 = 3</td></tr><tr><td>2</td><td>3</td><td>p >= 0: x+1, y+1</td><td>(4, 3)</td><td>3 - 4 = -1</td></tr><tr><td>3</td><td>-1</td><td>p < 0: x+1</td><td>(5, 3)</td><td>-1 + 6 = 5</td></tr><tr><td>4</td><td>5</td><td>p >= 0: x+1, y+1</td><td>(6, 4)</td><td>5 - 4 = 1</td></tr></table>The result is the same as DDA, but computed with integers only. For slopes greater than 1, swap the roles of x and y; for negative slopes or other octants, change the signs of the increments.</td></tr>
<tr><td>7</td><td>Explain the DDA circle drawing algorithm.</td><td>A circle centred at the origin is $x^2 + y^2 = r^2$. The DDA (incremental) method moves the point along the circle by repeatedly rotating it by a small angle, using the previous point to find the next:<pre>x(n+1) = x(n) + e * y(n)
y(n+1) = y(n) - e * x(n+1)   (uses the updated x)

start at (x0, y0) = (0, r)
e = 2^-k   where 2^(k-1) <= r < 2^k
stop when the point returns to the start (x0, y0)
add the centre (xc, yc) to every point before plotting</pre>
Read the radius r and the centre (xc, yc).
Choose k so that $2^{k-1} <= r < 2^k$ and set $e = 2^{-k}$.
Set x = 0, y = r.
Repeat: plot (round(x) + xc, round(y) + yc); compute new x and y with the formulas above; stop when back at the start.
Using e as a power of two means the multiplications can be done by shifts. The method is simple, but it uses floating-point arithmetic and its accumulated error makes the circle drift slightly. Using the eight-way symmetry of the circle (below) can reduce the work.</td></tr>
</table>

<table>
<tr><th>S.No</th><th>Question</th><th>Answer</th></tr>
<tr><td>8</td><td>Explain Bresenham's circle drawing algorithm.</td><td>A circle has eight-way symmetry: if (x, y) lies on a circle centred at the origin, so do (+-x, +-y) and (+-y, +-x). So only one octant (from x = 0 to x = y) needs to be calculated, and the other seven points are obtained by reflection. Bresenham's method uses an integer decision parameter d to choose between pixel (x+1, y) and (x+1, y-1).<pre>
void bresCircle(int xc, int yc, int r) {
    int x = 0, y = r, d = 3 - 2 * r;
    while (x <= y) {
        plot8(xc, yc, x, y);   // 8 symmetric points
        if (d < 0)
            d += 4 * x + 6;
        else {
            d += 4 * (x - y) + 10;
            y--;
        }
        x++;
    }
}
</pre>Example: r = 5, d0 = 3 - 10 = -7.<table>
<tr><th>Plotted (x, y)</th><th>d before</th><th>Update</th><th>d after</th></tr>
<tr><td>(0, 5)</td><td>-7</td><td>d < 0: d + 4x + 6</td><td>-1</td></tr>
<tr><td>(1, 5)</td><td>-1</td><td>d < 0: d + 4x + 6</td><td>9</td></tr>
<tr><td>(2, 5)</td><td>9</td><td>d >= 0: d + 4(x - y) + 10, y = y - 1</td><td>7</td></tr>
<tr><td>(3, 4)</td><td>7</td><td>d >= 0: d + 4(x - y) + 10, y = y - 1</td><td>13 (then x > y, stop)</td></tr>
</table></td></tr>
<tr><td>9</td><td>Explain the midpoint circle drawing algorithm.</td><td>The midpoint algorithm uses the circle function $f(x, y) = x^2 + y^2 - r^2$, which is negative inside the circle, zero on it and positive outside. At each step it evaluates f at the midpoint between the two candidate pixels (x+1, y) and (x+1, y-1); the sign tells which pixel is nearer to the circle.
Start at (0, r). Initial decision parameter p0 = 1 - r.
If p < 0: the midpoint is inside, choose (x+1, y); p = p + 2x + 3 (with the old x), that is p + $2x_{new}$ + 1.
Else: choose (x+1, y-1); p = p + $2x_{new}$ + 1 - $2y_{new}$.
Plot the eight symmetric points each time, and repeat while x < y.
Example: r = 10, p0 = 1 - 10 = -9. The points in the first octant, from (0, 10), are:<pre>
(0,10) (1,10) (2,10) (3,10) (4,9) (5,9) (6,8) (7,7)
p values: -9, -6, -1, 6, -3, 8, 5, 6 (stop when x >= y)
</pre>The method gives the same pixels as Bresenham's circle algorithm and uses integers only.</td></tr>
<tr><td>10</td><td>What is a polygon? Explain its types: convex, concave and complex.</td><td>A polygon is a closed figure bounded by a sequence of straight line segments (edges) joined end to end at points called vertices.<table>
<tr><th>Type</th><th>Description</th><th>Test</th></tr>
<tr><td>Convex</td><td>Every line segment between two points of the polygon lies inside it; all interior angles are less than 180 degrees</td><td>Cross products of consecutive edges all have the same sign</td></tr>
<tr><td>Concave</td><td>At least one interior angle is greater than 180 degrees; some segment between two points goes outside it</td><td>Cross products have mixed signs</td></tr>
<tr><td>Complex (self-intersecting)</td><td>Edges cross each other, so the boundary intersects itself (for example a star drawn in one stroke)</td><td>Some pair of non-adjacent edges intersects</td></tr>
</table>Convex and concave polygons together are called simple polygons.</td></tr>
</table>

<table>
<tr><th>S.No</th><th>Question</th><th>Answer</th></tr>
<tr><td>11</td><td>What is the inside test in polygons?</td><td>The inside test (point-in-polygon test) decides whether a point lies inside a polygon. It is needed for filling and for complex polygons whose interior is not obvious.
Odd-even (even-odd) rule: draw a ray from the point to infinity and count how many polygon edges it crosses. An odd count means inside, an even count means outside.
Non-zero winding number rule: count how many times the boundary winds around the point (edges crossing the ray from left to right add +1, right to left add -1). The point is inside if the winding number is not zero.
For convex polygons it is enough to check that the point is on the same side of every edge.
The two rules agree for simple polygons but differ for self-intersecting ones; for the middle pentagon of a five-point star, the odd-even rule treats it as outside while the winding rule treats it as inside.</td></tr>
<tr><td>12</td><td>Explain the flood fill and seed fill algorithms.</td><td>Both are seed fill methods: they start from a seed pixel inside the region and spread outwards to the neighbouring pixels. They can fill regions that are defined pixel by pixel, such as irregular shapes.<table><tr><th>Boundary fill</th><th>Flood fill</th></tr><tr><td>Region has a single boundary colour</td><td>Region has a single interior colour</td></tr><tr><td>Spreads until the boundary colour is met</td><td>Replaces every connected pixel of the old colour</td></tr><tr><td>Interior can contain several colours</td><td>Interior must be one colour</td></tr></table><pre><code>void boundaryFill(int x, int y, int fill, int boundary) {
    int c = getpixel(x, y);
    if (c != boundary && c != fill) {
        putpixel(x, y, fill);
        boundaryFill(x + 1, y, fill, boundary);
        boundaryFill(x - 1, y, fill, boundary);
        boundaryFill(x, y + 1, fill, boundary);
        boundaryFill(x, y - 1, fill, boundary);
    }
}
void floodFill(int x, int y, int newCol, int oldCol) {
    if (getpixel(x, y) == oldCol) {
        putpixel(x, y, newCol);
        floodFill(x + 1, y, newCol, oldCol);   // same four calls as above
        floodFill(x - 1, y, newCol, oldCol);
        floodFill(x, y + 1, newCol, oldCol);
        floodFill(x, y - 1, newCol, oldCol);
    }
}</code></pre>
4-connected fill uses the four neighbours (left, right, up, down); 8-connected fill also uses the diagonals and can fill thin diagonal regions, but can leak through a one-pixel diagonal boundary.
The recursive version can overflow the stack on large regions; practical versions use an explicit stack or fill whole scan-line spans.</td></tr>
</table>

S.No	Question	Answer
13	**Explain the scan-line polygon fill algorithm.**	The scan-line algorithm fills a polygon **one horizontal line at a time**, from the bottom to the top. 1. For the scan line y, find the intersections of the line with all polygon edges. 2. Sort the intersection x-coordinates in increasing order. 3. Pair them up (1st-2nd, 3rd-4th, ...) by the odd-even rule and fill the pixels between the pairs. 4. Move to the next scan line y + 1. • **Edge table (ET)** stores every non-horizontal edge by its lower y: y_{max}, x at y_{min}, and 1/m. • **Active edge table (AET)** holds only the edges crossed by the current scan line. When the scan line goes up by one, each edge's intersection is updated incrementally: x = x + 1/m. Edges are removed when y exceeds y_{max}. • **Special cases:** horizontal edges are ignored; at a vertex shared by two edges, count the intersection once if the edges are on opposite sides of the scan line, and twice (or not at all) if they are on the same side, usually by shortening one edge by one scan line. Edge coherence (the next scan line's intersection differs by a constant 1/m) is what makes the algorithm fast.

14	**What is windowing and clipping?**	A **window** is the rectangular area of the **world** coordinate system that we want to see. A **viewport** is the rectangular area of the **device (screen)** in which that part is displayed. **Windowing** (the viewing transformation) maps the window to the viewport.
		`xv = xvmin + (xw - xwmin) * sx  sx = (xvmax - xvmin) / (xwmax - xwmin)`
		`yv = yvmin + (yw - ywmin) * sy  sy = (yvmax - yvmin) / (ywmax - ywmin)`
		Clipping is the process of identifying which parts of a picture lie inside the window and discarding the parts outside it, so that nothing outside the region is drawn. It can be done for points, lines, polygons, curves and text. Clipping saves time because invisible parts are not processed further.
		Example: window (0, 0)-(10, 10) mapped to viewport (0, 0)-(200, 100): sx = 20, sy = 10, so the world point (5, 4) maps to (100, 40).

<table>
<tr><th>S.No</th><th>Question</th><th>Answer</th></tr>
<tr><td>15</td><td>Explain the Cohen-Sutherland line clipping algorithm.</td><td>The algorithm quickly accepts or rejects most lines using a 4-bit region code for each end point, and clips only the lines that need it.
<table><tr><th>Bit (value)</th><th>Set when the point is</th><th>Condition</th></tr><tr><td>Bit 4 (8)</td><td>Above the window</td><td>y > ymax</td></tr><tr><td>Bit 3 (4)</td><td>Below the window</td><td>y < ymin</td></tr><tr><td>Bit 2 (2)</td><td>Right of the window</td><td>x > xmax</td></tr><tr><td>Bit 1 (1)</td><td>Left of the window</td><td>x < xmin</td></tr></table>
Compute the region codes of both end points. The inside region is 0000.
Trivially accept if both codes are 0000 (the line is fully inside).
Trivially reject if the bitwise AND of the two codes is not 0000 (both ends are on the outer side of the same boundary).
Otherwise take an end point that is outside, and replace it with the intersection of the line with a window boundary it violates. Recompute its code and go back to step 2.
<pre>m = (y2 - y1) / (x2 - x1)
left   x = xmin : y = y1 + m * (xmin - x1)
right x = xmax : y = y1 + m * (xmax - x1)
bottom y = ymin : x = x1 + (ymin - y1) / m
top   y = ymax : x = x1 + (ymax - y1) / m</pre>Example: window (0, 0)-(10, 10); line A(-4, 2) to B(6, 12). Code of A = 0001 (left), code of B = 1000 (top). AND = 0000, so the line is not rejected. m = 1. Clip A at x = 0: y = 2 + 1 x (0 + 4) = 6, so A' = (0, 6). Clip B at y = 10: x = 6 + (10 - 12) / 1 = 4, so B' = (4, 10). Both new codes are 0000, so the visible part is (0, 6) to (4, 10).</td></tr>
<tr><td>16</td><td>Explain the Sutherland-Hodgeman polygon clipping algorithm.</td><td>The polygon is clipped against one window boundary at a time (left, right, bottom, top). The output vertex list of one stage becomes the input of the next. For each edge from vertex S to vertex P of the polygon, four cases are possible:
<table><tr><th>Case</th><th>S</th><th>P</th><th>Output added</th></tr><tr><td>1</td><td>inside</td><td>inside</td><td>P</td></tr><tr><td>2</td><td>inside</td><td>outside</td><td>Intersection point I</td></tr><tr><td>3</td><td>outside</td><td>outside</td><td>Nothing</td></tr><tr><td>4</td><td>outside</td><td>inside</td><td>Intersection point I, then P</td></tr></table>
Take the first window boundary and an empty output list.
Go around the polygon edge by edge (including the closing edge) and apply the four cases.
Use the output list as the new polygon and repeat with the next boundary.
After the four boundaries, the final list is the clipped polygon.
Works well for convex polygons, and for any polygon against a convex window.
Limitation: a concave polygon may produce extra connecting edges along the window boundary; these can be removed by splitting into convex parts or by using Weiler-Atherton clipping.</td></tr>
</table>

S.No	Question	Answer

17	**Case study: real-time map rendering using line and circle drawing algorithms; 3D pipeline and polygonal modelling in real-world applications.**	**(a) Real-time map rendering** (for example a navigation app).
		• Roads and boundaries are stored as polylines and drawn with **Bresenham's line algorithm**, because it uses integer arithmetic only and is fast enough to redraw the map every frame while panning.
		• Points of interest, the user's position marker and accuracy or search-radius circles are drawn with the **midpoint or Bresenham circle algorithm**, using eight-way symmetry to calculate only one octant.
		• Only the part of the map inside the screen window is processed: lines are clipped with **Cohen-Sutherland** and area features such as parks and lakes with **Sutherland-Hodgeman**, and then mapped to the screen by the window-to-viewport transformation.
		• Land and water areas are filled with **scan-line fill**, and level-of-detail (fewer lines at low zoom) keeps the frame rate high.
		(b) 3D pipeline and polygonal modelling (games, CAD, film, medical visualisation).
		`Model (triangle mesh) -> Modelling transform (local -> world)`
		`-> Viewing transform (world -> camera)`
		`-> Projection (3D -> 2D, perspective)`
		`-> Clipping (against view volume)`
		`-> Viewport mapping (to screen)`
		`-> Rasterisation (scan conversion) -> Shading, texturing, depth test`
		`-> Frame buffer -> Display`
		• Objects are approximated by **polygonal meshes** (mostly triangles) because triangles are always flat and convex, so they are easy to transform, clip and fill.
		• Every vertex goes through the same matrix transformations; the rasteriser then uses line and polygon scan conversion to turn each triangle into pixels, and a z-buffer decides which surface is visible.
		• Graphics hardware (GPUs) implements this pipeline in parallel, which allows millions of triangles to be drawn every frame.

UNIT V - 2D and 3D Transformations and Projections

<table>
<tr><th>S.No</th><th>Question</th><th>Answer</th></tr>
<tr><td>1</td><td>What is a 2D transformation? Explain its types: translation, scaling, rotation and shear.</td><td>A 2D transformation changes the position, size, orientation or shape of an object by applying a rule to the coordinates of its points. To combine transformations by matrix multiplication, points are written in homogeneous coordinates (x, y, 1), so every transformation becomes a 3 x 3 matrix.
<pre>Translation (tx, ty)   Scaling (sx, sy)   Rotation (angle a)
1 0 tx		sx 0 0		cos -sin 0
0 1 ty		0 sy 0		sin cos 0
0 0 1		0 0 1		0 0 1

Shear in x (shx)   Shear in y (shy)
| 1 shx 0 | | 1   0   0 |
| 0 1   0 | | shy 1   0 |
| 0 0   1 | | 0   0   1 |</pre><table><tr><th>Type</th><th>Effect</th><th>Equations</th></tr><tr><td>Translation</td><td>Moves every point by the same distance in a straight line (rigid)</td><td>x' = x + tx, y' = y + ty</td></tr><tr><td>Scaling</td><td>Enlarges or shrinks about the origin; sx = sy gives uniform scaling</td><td>x' = x.sx, y' = y.sy</td></tr><tr><td>Rotation</td><td>Turns the object by an angle (counter-clockwise positive) about the origin</td><td>x' = x cos(a) - y sin(a), y' = x sin(a) + y cos(a)</td></tr><tr><td>Shear</td><td>Slants the shape by displacing one coordinate in proportion to the other</td><td>x' = x + shx.y, y' = y (x-shear)</td></tr></table>Examples: translating (2, 3) by (4, -1) gives (6, 2). Scaling (2, 3) by (2, 2) gives (4, 6). Rotating (1, 0) by 90 degrees gives (0, 1). Reflection is a special scaling with a negative factor (for example sy = -1 reflects about the x-axis).</td></tr>
<tr><td>2</td><td>Explain rotation about an arbitrary point in 2D.</td><td>The standard rotation matrix turns points about the origin, so to rotate about a pivot point (xp, yp) we move the pivot to the origin, rotate, and move it back.
Translate by (-xp, -yp) so that the pivot is at the origin.
Rotate by the angle theta about the origin.
Translate back by (xp, yp).
<pre>M = T(xp, yp) . R(theta) . T(-xp, -yp)   (apply right to left)

        | cos -sin   xp(1 - cos) + yp.sin |
    M = | sin   cos   yp(1 - cos) - xp.sin |
        | 0   0   1 |</pre>Example: rotate the point (4, 3) by 90 degrees about (1, 1). Translate: (3, 2). Rotate: (x cos - y sin, x sin + y cos) = (-2, 3). Translate back: (-1, 4). Matrix multiplication is not commutative, so the order of the three steps matters.</td></tr>
</table>

S.No	Question	Answer

3	**What is a 3D transformation? Explain translation, scaling, rotation and shear.**	3D transformations act on points (x, y, z) represented in homogeneous coordinates (x, y, z, 1) and are 4 x 4 matrices. The same ideas as in 2D apply, with one more axis.
		`Translation (tx,ty,tz)  Scaling (sx,sy,sz)`
		`| 1 0 0 tx |  | sx 0 0 0 |`
		`| 0 1 0 ty |  | 0 sy 0 0 |`
		`| 0 0 1 tz |  | 0 0 sz 0 |`
		`| 0 0 0 1 |  | 0 0 0 1 |`
		`Rotation about Z  Rotation about X  Rotation about Y`
		`| c -s 0 0 |  | 1 0 0 0 |  | c 0 s 0 |`
		`| s c 0 0 |  | 0 c -s 0 |  | 0 1 0 0 |`
		`| 0 0 1 0 |  | 0 s c 0 |  | -s 0 c 0 |`
		`| 0 0 0 1 |  | 0 0 0 1 |  | 0 0 0 1 |`
		`(c = cos(a), s = sin(a), counter-clockwise looking toward the origin)`
		`Shear along Z (a, b)`
		`| 1 0 a 0 |  x' = x + a.z`
		`| 0 1 b 0 |  y' = y + b.z`
		`| 0 0 1 0 |  z' = z`
		`| 0 0 0 1 |`
		• **Translation:** moves the object by (tx, ty, tz).
		• **Scaling:** changes the size along each axis; non-uniform scaling distorts the shape.
		• **Rotation:** in 3D a rotation needs an **axis**. The axis coordinate stays unchanged (rotation about Z leaves z fixed).
		• **Shear:** shifts two coordinates in proportion to the third, so the shape slants while the third coordinate is unchanged.

4	**Explain rotation about an arbitrary axis in 3D.**	The rotation matrices only turn objects about the coordinate axes. To rotate by an angle theta about an arbitrary line through the point P with unit direction vector (a, b, c), the axis is first moved onto a coordinate axis (the Z-axis), the rotation is done there, and the moves are undone in reverse order.
		1. **Translate** so that the axis passes through the origin: T(-P).
		2. **Rotate about X** by angle alpha to bring the axis into the XZ-plane: with d = sqrt(b^2 + c^2), cos(alpha) = c / d and sin(alpha) = b / d.
		3. **Rotate about Y** by angle beta to bring the axis onto the Z-axis: cos(beta) = d and sin(beta) = -a.
		4. **Rotate about Z** by the required angle theta: Rz(theta).
		5. Apply the inverse of step 3, then the inverse of step 2, then **translate back** by P.
		`M = T(P) . Rx(-alpha) . Ry(-beta) . Rz(theta) . Ry(beta) . Rx(alpha) . T(-P)`
		`(applied right to left)`
		If d = 0 the axis is already along X, and the X rotation is skipped. A special case is when the axis is parallel to a coordinate axis: then only translate, rotate about that axis, and translate back.

5	**What is a projection? Why is it used in computer graphics?**	A projection maps points of a **3D** object onto a **2D** projection (view) plane by passing straight lines, called **projectors**, through every point of the object and finding where they meet the plane. • Screens and paper are 2D, so a 3D scene must be converted to 2D before it can be shown. • Different projections give different results: precise measurable drawings (engineering) or realistic views (games, films). • It is a stage of the graphics pipeline, applied after the viewing transformation and before clipping to the viewport. The two main classes are **parallel** projection (projectors are parallel) and **perspective** projection (projectors meet at a centre of projection).

<table>
<tr><th>S.No</th><th>Question</th><th>Answer</th></tr>
<tr><td>6</td><td>Differentiate between parallel and perspective projections.</td><td>
<table>
<tr><th>Parallel projection</th><th>Perspective projection</th></tr>
<tr><td>Projectors are parallel; centre of projection is at infinity</td><td>Projectors meet at a finite centre of projection (the eye)</td></tr>
<tr><td>Size does not change with distance</td><td>Distant objects look smaller (perspective foreshortening)</td></tr>
<tr><td>Parallel lines remain parallel</td><td>Parallel lines converge at vanishing points</td></tr>
<tr><td>Preserves true dimensions and angles along some axes</td><td>Does not preserve lengths or angles</td></tr>
<tr><td>Looks less realistic</td><td>Looks realistic, like human vision or a camera</td></tr>
<tr><td>Used in engineering and architectural drawings</td><td>Used in games, films, VR and visualisation</td></tr>
</table>
<pre>Perspective: view plane z = d, eye at the origin
   x' = x . d / z   y' = y . d / z

| 1 0 0   0 | (x, y, z, 1) -> (x, y, z, z/d), then divide by w = z/d
| 0 1 0   0 |
| 0 0 1   0 | Orthographic projection on z = 0:   x' = x, y' = y
| 0 0 1/d 0 |</pre>
</td></tr>
<tr><td>7</td><td>Explain the types of parallel projections: oblique (cavalier, cabinet) and orthographic (isometric, diametric, trimetric).</td><td>
<table>
<tr><th>Class</th><th>Projectors</th><th>Types</th></tr>
<tr><td>Orthographic</td><td>Perpendicular to the view plane</td><td>Multiview (front, top, side) and axonometric (isometric, diametric, trimetric)</td></tr>
<tr><td>Oblique</td><td>Parallel to each other but not perpendicular to the view plane</td><td>Cavalier, cabinet</td></tr>
</table>
Orthographic (axonometric) projections turn the object so that three faces are visible; they differ in how much each axis is shortened.
<table>
<tr><th>Type</th><th>Foreshortening of the three axes</th><th>Notes</th></tr>
<tr><td>Isometric</td><td>Equal on all three axes (about 0.816)</td><td>Projected axes are 120 degrees apart; the direction of projection makes equal angles with the three axes</td></tr>
<tr><td>Diametric</td><td>Equal on two axes, different on the third</td><td>Two angles between the axes are equal</td></tr>
<tr><td>Trimetric</td><td>Different on all three axes</td><td>All angles between the axes are different; the most general and flexible</td></tr>
</table>
Oblique projections keep faces parallel to the view plane true in shape and size, and draw the receding (depth) lines at an angle.
<table>
<tr><th>Type</th><th>Scale of depth lines</th><th>Angle of projectors with the plane</th><th>Result</th></tr>
<tr><td>Cavalier</td><td>Full length (factor 1)</td><td>tan(a) = 1, a = 45 degrees</td><td>Looks elongated in depth</td></tr>
<tr><td>Cabinet</td><td>Half length (factor 1/2)</td><td>tan(a) = 2, a = 63.4 degrees</td><td>Looks more natural</td></tr>
</table>
<pre>x' = x + z . L1 . cos(phi)   L1 = 1 / tan(a)
y' = y + z . L1 . sin(phi)   Cavalier: L1 = 1, Cabinet: L1 = 1/2</pre>
</td></tr>
</table>

<table>
<tr><th>S.No</th><th>Question</th><th>Answer</th></tr>
<tr><td>8</td><td>Explain 1-point, 2-point and 3-point perspective projections.</td><td>In perspective projection, any set of parallel lines that is not parallel to the view plane appears to converge at a vanishing point. The number of principal vanishing points equals the number of principal axes that the view plane cuts.
<table>
<tr><th>Type</th><th>View plane</th><th>Vanishing points</th><th>Typical use</th></tr>
<tr><td>1-point</td><td>Parallel to two principal axes (cuts only the Z-axis); one face of the object faces the viewer</td><td>One, on the horizon; lines parallel to the Z-axis converge</td><td>Roads, corridors, interiors</td></tr>
<tr><td>2-point</td><td>Cuts two axes (for example X and Z); the object is turned about the vertical axis; vertical lines stay vertical</td><td>Two, on the horizon</td><td>Buildings seen from a corner</td></tr>
<tr><td>3-point</td><td>Cuts all three axes; the viewer looks up or down at the object</td><td>Three; vertical lines also converge</td><td>Skyscrapers from ground level or from above</td></tr>
</table>
More vanishing points mean a more dramatic and more realistic view, but the drawing and the computation are more complicated.</td></tr>
<tr><td>9</td><td>Compare 2D and 3D transformations.</td><td>
<table>
<tr><th>Property</th><th>2D transformation</th><th>3D transformation</th></tr>
<tr><td>Point</td><td>(x, y), homogeneous (x, y, 1)</td><td>(x, y, z), homogeneous (x, y, z, 1)</td></tr>
<tr><td>Matrix size</td><td>3 x 3</td><td>4 x 4</td></tr>
<tr><td>Translation parameters</td><td>tx, ty</td><td>tx, ty, tz</td></tr>
<tr><td>Scaling factors</td><td>sx, sy</td><td>sx, sy, sz</td></tr>
<tr><td>Rotation</td><td>About a point (the origin or a pivot); one angle</td><td>About an axis (X, Y, Z or arbitrary); three basic matrices</td></tr>
<tr><td>Shear</td><td>x-shear and y-shear</td><td>Along X, Y and Z, each affecting the other two coordinates</td></tr>
<tr><td>Reflection</td><td>About a line or the origin</td><td>About a plane</td></tr>
<tr><td>Order of composition</td><td>Matters (not commutative), except for rotations in the same plane</td><td>Matters, even for rotations about different axes</td></tr>
<tr><td>Display</td><td>Directly on the screen</td><td>Needs projection onto a 2D plane</td></tr>
<tr><td>Computation</td><td>Simple and fast</td><td>More expensive (hardware GPUs are used)</td></tr>
</table>
</td></tr>
</table>